PRISM: Unleashing the Power of Large-Scale LLM Training on Superchips

Abstract

The emergence of Superchips represents a significant advancement in next-generation AI hardware. These Superchips employ a tightly coupled heterogeneous architecture that integrates GPU and CPU on the same die, which offers unprecedented computational power. However, there has been scant research investigating how LLM training benefits from this new architecture. In this work, for the first time, we study LLM training solutions based on offloading for Superchips. We observe important differences between Superchips and traditional loosely-coupled GPU-CPU architecture, which necessitate revisiting prevailing assumptions about offloading. Based on that, we present PRISM, a Superchip-centric offloading system that simultaneously uses Hopper GPU, Grace CPU, and NVLink-C2C interconnect more efficiently. PRISM accomplishes this via a combination of techniques, such as adaptive weight offloading, bucketization repartitioning, Superchip-aware casting, speculative execution, and a highly optimized Adam optimizer for Grace CPUs. Our evaluation of PRISM on NVIDIA GH200 demonstrates up to 2.5× throughput improvement compared to state-of-the-art offloading-based systems, enabling training of up to 25B model on a single Superchip while achieving high training throughput. We also extend PRISM with ZeRO-style data parallelism and DeepSpeed-Ulysses sequence parallelism, enabling training of 13B model with sequence lengths up to 1 million tokens on 8 GH200 while achieving 55% MFU.

1 Introduction

The impressive capabilities of Large Language Models (LLMs) have spurred significant research and industry efforts [1, 32, 42, 62, 67] to optimize the resource and time costs of training state-of-the-art LLMs. A large fraction of this work is motivated by the *memory wall* challenge of scaling LLMs [13] – the disparity between exponential growth in model sizes and relatively slower increase in hardware memory capacity and bandwidth [23, 51]. Consequently, training LLMs is often constrained by limited GPU memory.

When LLM training exceeds a single GPU's memory capacity, practitioners often employ a variety of distributed techniques, such as ZeRO-style data parallelism [50], tensor-slicing parallelism [59], pipeline parallelism [17, 35, 36], and sequence parallelism [21, 28], to shard model and data states across multiple GPUs. However, the large number of GPUs required by these methods poses a significant bottleneck for researchers and institutions with modest GPU budgets.

Yet, enabling training of LLMs with a limited number of GPUs is critical in many scenarios. Beyond pre-training, LLMs often reach their full potential through additional post-training phases, such as long context extension [8, 32], continuous training [27, 56, 60], supervised fine-tuning [6], instruction tuning [69], and human preference alignment techniques such as RLHF [43] and DPO [49], all of which require model training, even when users have significantly fewer GPUs than used for pre-training.

Among different approaches, offloading techniques have become a promising solution to reduce GPU requirements for large-scale LLM training [31, 51, 54, 71]. These methods offload a portion of the model states and computation from GPU to CPU, effectively utilizing both to enable large-scale LLM training without excessive GPU usage. As a result, offloading is supported by major DL training frameworks such as PyTorch-FSDP [72] and DeepSpeed [51, 54].

While showing promising results, existing offloading solutions are based on one key assumption: the GPU is an accelerator connected to the CPU via PCIe, which has limited communication bandwidth (e.g., 32GB/s for PCIe-Gen4). Due to this bandwidth limitation, prior work primarily focuses on optimizing data transfers between GPU and CPU to avoid PCIe becoming a major performance bottleneck [47, 51, 54]. However, hardware vendors have embarked on the development of a new class of high-end tightly coupled hardware architecture in their product roadmaps [40], e.g., NVIDIA Grace Hopper GH200 [41] and AMD MI300A [2], which herald a paradigm shift in deep learning infrastructure.

Take NVIDIA's Grace Hopper Superchip (GH200) as an example. It integrates a Hopper H100 GPU and an ARM-based Grace CPU on the same die through the NVLink-Chip2Chip (C2C) interconnect, with GPU-CPU bandwidth reaching a staggering 900 GB/s—30× increase over traditional PCIe connections. Moreover, multiple Superchips can be connected to form a large-scale tightly coupled system, which offers fast and low latency interaction between different classes of chiplets. Given the tightly coupled hardware architecture, recent work has started benchmarking Superchips [10, 18, 57] and how to best program them [68]. However, efficient utilization of Superchips for offloading-based LLM training has not yet been studied in depth.

In this work, we investigate software optimizations to train LLMs in the era of Superchips. We find that existing offloading methods, designed under the assumption of limited PCIe bandwidth, perform suboptimally on Superchips. In particular, off-the-shelf solutions severely underutilize the Hopper GPU, Grace CPU, and C2C bandwidth. For example, prior work often adopts a greedy algorithm for data transfer in mixed-precision training, transferring only low-precision

```
import torch                          import torch
...                                   import prism
...                                   ...
model = BuildModel(config)            model = BuildModel(config)
optimizer = Optimizer(model)          optimizer = Optimizer(model)
...                                   model = prism.initialize(
...                                           model, optimizer)
for batch in batches:                 for batch in batches:
    loss = model(batch)                   loss = model(batch)
    loss.backward()                       model.backward()
    optimizer.step()                      model.step()
```

Figure 1. PRISM can be enabled with a few lines of change. The code on left shows a standard training pipeline, while the right shows the same pipeline with PRISM.

data between CPU and GPU to minimize PCIe communication volume in order to maximize training efficiency. Our detailed analysis reveals that these common assumptions do not hold true for Superchips.

Contribution. This paper addresses a pressing need to harness the power of Superchips for training LLMs. Concretely, we take a leap forward and introduce PRISM, a Superchip-centric offloading system that delivers excellent training performance on NVIDIA GH200 Superchips. PRISM accomplishes this via a combination of techniques, such as *adaptive weight-stationary and weight-flow offloading* (§ 4.2), *fine-grained bucketization repartitioning* (§ 4.3), *speculation-then-validation* support to overlap compute and communication (§ 4.4), *Superchip-aware typecasting for mixed-precision training* (§ 4.5), and *highly optimized GraceAdam* for Grace Arm CPUs (§ 4.6). To our knowledge, this is the first work that systematically optimizes LLM training for Superchip architectures, maximizing the utilization of Hopper GPU, Grace CPU, and C2C interconnect, simultaneously.

We validate our design through comprehensive experiments. PRISM achieves a 2.5× throughput improvement over state-of-the-art offloading solutions and outperforms GPU-only approaches across all tested model sizes, which challenges the conventional wisdom that offloading often comes at the cost of performance penalty. PRISM also enables 25B-parameter model training on a single Superchip, which is 7× larger than GPU-only solutions.

In addition, we validate our design by integrating PRISM with ZeRO-style data parallelism [50], which allows PRISM to extend to multi-Superchip training settings. Specifically, PRISM enables LLM training with 50B parameters using only four Superchips, which is 2.5× larger than the largest model trainable with ZeRO-Offload and FSDP-Offload, while achieving up to 3× higher training throughput.

We also extend PRISM to support long-sequence training, termed PRISM-Ulysses, via integration with Ulysses-style sequence parallelism [21]. This unlocks new possibilities for training or fine-tuning LLMs with long sequence lengths. Specifically, PRISM-Ulysses supports training sequences 8× longer than Ulysses and enables the training of a 13B model

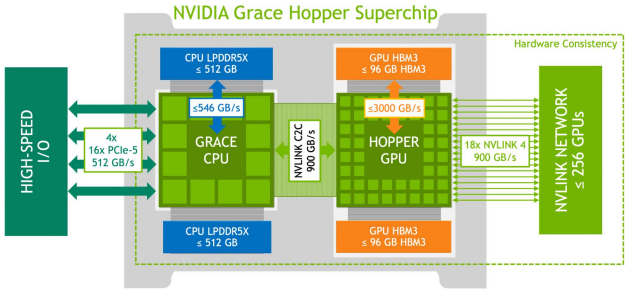

Figure 2. Grace Hopper GH200 Superchip hardware overview [39]. The Grace CPU, equipped with a 500 GB/s DDR interface, is shown on the left, and the Hopper GPU, with 4000 GB/s HBM, is depicted on the right. The two are interconnected via NVLINK-C2C at 900 GB/s.

with up to a million tokens using only 8 GH200 Superchips, while achieving 55% MFU. To enhance usability, PRISM has been integrated with DeepSpeed [52], a popular open-source DL training library. Users can enable PRISM through a few lines, as illustrated in Fig. 1, without model code refactoring.

2 Background and Related Work

2.1 The Emergence of Superchips

GPUs were initially developed as accelerators loosely-coupled with CPUs. Driven by the need to support the next generation of AI systems, NVIDIA introduced GH200 Grace Hopper Superchip [41], a groundbreaking processor that integrates a tightly-coupled Hopper GPU and Grace CPU on the same die with high-bandwidth NVLink Chip-2-Chip (C2C) interconnect, as shown in Fig. 2. Recently, NVIDIA also announced GB200 [40], the next-generation Superchip that has a tightly-coupled Blackwell GPU with Grace CPU to deliver unprecedented performance and efficiency for AI and other advanced computing fields. Other hardware vendors, such as AMD, have released AMD Instinct MI300A Superchip [2].

Despite being a new architecture, Superchips have already been adopted in large-scale supercomputing clusters worldwide, including France's EXA1-HE [20], Poland's Helios [15], Switzerland's Alps [63], Germany's JUPITER [22], the United States' NCSA [37], and Japan's Miyabi [19]. Notably, the Alps supercomputer has integrated over 2,000 GH200 nodes and Isambard-AI supercomputer plans to deploy over 5,000 GH200 nodes to enhance computational capabilities. The GH200 Superchip is also available through cloud service providers such as Amazon Web Services [58], Microsoft Azure [5], and Lambda [25], offering flexible options for organizations seeking high-performance computing resources.

Given the ever-growing adoption, Superchips have attracted significant attention with active research on benchmarking their hardware characteristics [10, 18, 57] and their programmability [68]. However, few studies have been done to investigate how to effectively leverage it for LLM training.

2.2 LLM Training on Heterogeneous Hardware

Modern LLM training is highly memory-intensive due to the large number of parameters and the significant memory required for model states like gradients and optimizer states [24, 30]. As an example, during mixed precision training [33], a model with Ψ parameters consumes a total of 16Ψ bytes of memory (e.g., 2Ψ parameters, 2Ψ gradients, and 12Ψ optimizer states). An NVIDIA H100 GPU with 96GB memory can only accommodate a model with up to 6B parameters.

To overcome the memory limitations, advanced distributed training techniques such as tensor parallelism (TP) [59], pipeline parallelism (PP) [17, 35], ZeRO-style data parallelism (ZeRO-DP) [50], 3D parallelism [36] have been developed. Despite their strong capabilities, all of these methods rely on the aggregated memory of multiple GPUs to store the model states and residual states.

To reduce the number of GPUs needed, researchers have explored heterogeneous training that exploit the expansive memory capacity of CPUs to scale up model training [14, 16, 45, 53, 55, 66]. Offloading strategies like ZeRO-Offload [54] carefully divide the computation between the hardware types: GPU performs compute-intensive operations such as back-propagation, while CPU performs memory-intensive operations such as optimizer updates. ZeRO-Infinity further extends this approach to NVMe storage, enabling larger model training with limited GPU resources [51]. Deep-Optimizer-States extends ZeRO-Offload by fetching optimizer states from CPU to GPU and updating parameters in parallel across both devices, thus reducing optimizer step time in the critical path [31]. Similar to ZeRO-Infinity, SpeedLoader offloads both parameters and optimizer states to CPU memory, then dynamically fetches them to GPU layer by layer [71].

While demonstrating promising results, prior work often assume limited bandwidth between GPU and CPU via PCIe (e.g., <32GB/s). However, newer hardware, such as Superchips with 30× higher C2C bandwidth, challenges these assumptions. Consequently, adhering to the older design principles could lead to suboptimal performance, which motivates us to revisit the offloading design.

3 Challenges and Opportunities

NVIDIA's GH200 Grace Hopper Superchip represents a significant advancement in high-performance computing. However, this new hardware architecture poses challenges for existing off-the-shelf offloading solutions to *fully utilize all Superchip resources, including the Hopper GPU, C2C bandwidth, simultaneously*, for the following reasons:

Superchip ≠ GPU + CPU. Before the advent of Superchip, the connection between CPUs and GPUs relied on PCI Express (PCIe). For instance, as shown in Table 1, the DGX-A100 node [38], widely deployed within Meta for training the LLaMA model [64], comprises two AMD Rome CPUs and eight NVIDIA A100 GPUs, connected via PCIe 4.0 x16, delivering a bandwidth of 64 GB/s. Similarly, the DGX-2 node evaluated in ZeRO-Offload [54] contains two Intel Xeon CPUs and 16 NVIDIA V100 GPUs, with the CPU-GPU connection utilizing PCIe 3.0 x16, providing a bandwidth of 32 GB/s.

Table 1. Comparison of GPU node. DGX-2 is used in the ZeRO-Offload [54], DGX-A100 is used for LLaMa model training [64], and GH denotes a single Grace Hopper Superchip.

Node Arch	**DGX-2**	**DGX-A100**	**GH**
Hardware Setting	Intel Xeon+V100	AMD Roma+A100	GH200
CPU BW (GB/s)	100	150	**500**
C↔GPU BW (GB/s)	32	64	**900**
CPU Cores	24	64	72
CPU FLOPS (TFLOPS)	2.07	2.3	3.0
GPU FLOPS (TFLOPS)	125.0	312.0	990.0
GPU/CPU FLOPS	60.39	135.65	**330.0**

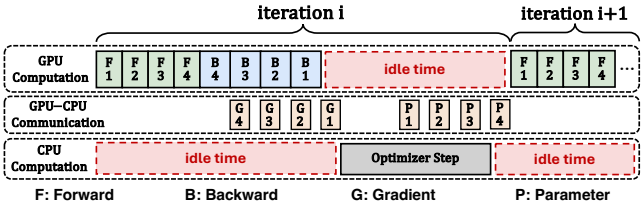

Figure 3. ZeRO-Offload computation idle time on both GPU and CPU side.

The Superchip, however, tightly couples an NVIDIA Hopper GPU with an NVIDIA Grace CPU on the same die via the NVLINK-C2C interconnect, delivering a staggering 900 GB/s bandwidth between GPU and CPU – 14× the standard PCIe Gen4 lanes. In addition, the 72-core Grace CPU significantly enhances compute capacity with 500 GB/s LPDDR5 high-bandwidth memory, providing not only additional memory space but also efficient handling of computation tasks.

Challenges in idle-free scheduling. Prior offloading-based solutions often suffer from GPU and CPU idle time due to GPU-CPU synchronization. Take ZeRO-Offload [54] as an example. As shown in Fig. 3, there are three primary constraints: 1) Global Gradient Synchronization: Gradient clipping and normalization require global information. Consequently, the CPU must wait until it has received all gradients from the GPUs before starting the optimizer step. This results in CPU idle time while the GPUs are busy with the backward pass. 2) Synchronized Parameter Updates: The GPU must wait for all FP16 parameters to return before initiating the forward pass of the next iteration. This synchronization is crucial to ensure that the forward pass uses updated parameters. 3) Limited Overlap of Computation and Communication: Even with small bucket sizes for transferring gradients to the CPU and updated parameters back to the GPU, complete overlap between computation and communication is unattainable (more details in § 4.3). Fig. 4 shows that 40-50% of the Hopper GPU is idle per training iteration.

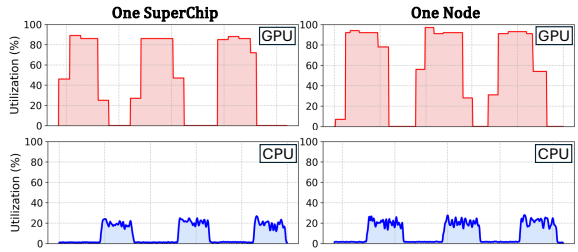

Figure 4. Prior offloading-based solutions cause idle time on both GPU and CPU side. We evaluate it using the largest model size that offloading-based solutions (ZeRO-Offload) can accommodate, along with the maximum batch size that prevents out-of-memory errors, on single Superchip and one node. The results show that during each training iteration, the GPU remains idle for 40-50% of the total execution time.

How can we reduce these inefficiencies for offloading-based solutions on Superchips?

Challenges in mixed-precision training on Superchips. Modern DL training often employs mixed precision training [33], which uses a mix of full and lower precisions to deliver significant computational speedup by executing operations in a lower precision format on matrix cores (e.g., TensorCore) as much as possible, while storing critical information in the full-precision to preserve model accuracy.

Mixed precision training makes offloading more complex because of additional casting operations in the computation graph. To enable efficient mixed-precision training through offloading, the conventional design principle was to adopt a greedy edge-cut algorithm [54]. As an example, ZeRO-Offload keeps all the optimizer states on the CPU, and moves the FP16 gradients to CPU during the backward pass and returns the updated FP16 parameter back to GPU during the optimizer step. This minimizes the communication volume between the GPU and CPU (e.g., 4Ψ), such that the PCIe bandwidth does not become a major bottleneck. However, our detailed performance analysis shows that this method is suboptimal for Superchip architectures (§ 4.5), which motivates us to revisit the decision of offloading-based mixed precision training.

There are other challenges such as Grace Arm CPU's lack of a high-performance Adam optimizer, and lacking of a clear solution towards scaling model sizes and long-sequence on multiple Superchips. In the following, we describe the design of PRISM, which specifically addresses all of the above challenges, eventually leading to a highly-optimized solution tailored for Superchips in many practical settings.

4 PRISM Design and Implementation

4.1 System Overview

PRISM takes the tightly-coupled Superchip architecture into account by considering a Superchip-aware DFG (SA-DFG). In SA-DFG, each vertex u still represents a tensor operator

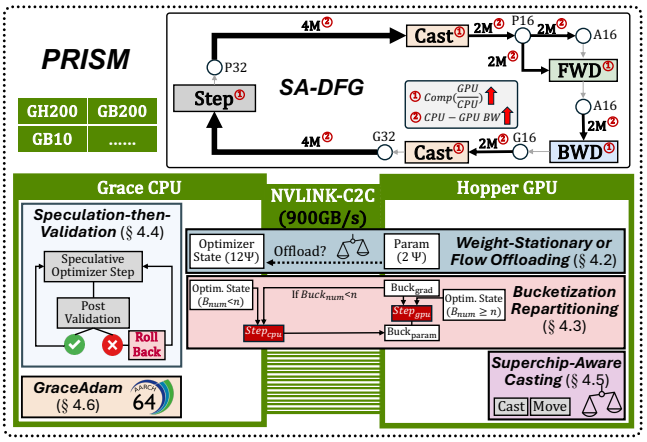

Figure 5. Overview of PRISM. At a high level, PRISM employs a Superchip-centric design that judiciously optimizes data placement, computation, and tensor migration between the Hopper GPU and Grace CPU, while fully utilizing the NVLink-C2C via a combination of techniques.

but it also reflects the computation cost that u would incur if it is assigned to the Hopper GPU or Grace CPU. The edge $u \longrightarrow v$ in SA-DFG reflects the communication cost that $u \longrightarrow v$ would incur if u and v are executed on a Hopper GPU or Grace CPU. An offload strategy then can be viewed as a two-way partition of SA-DFG.

Fig. 5 illustrates an overview of PRISM, a system specifically designed for training LLMs on Superchips by judiciously optimizing data placement, computation, and tensor migration between Hopper GPU and Grace CPU. This is achieved through a combination of techniques. First, it examines the amount of computation and communication associated with the operators in SA-DFG, and employs an adaptive weight-stationary and weight-flow policy to exploit heterogeneous compute efficiencies of Hopper GPU and Grace CPU (§ 4.2). Second, it adjusts the bucketization strategy and leverages fine-grained repartitioning to balance the workload among different resources on a Superchip (§ 4.3). Third, it introduces a novel speculation-then-validation algorithm to replace the conventional synchronization-then-execution scheme that enables effective overlapping between Hopper GPU and Grace CPU (§ 4.4). Fourth, it performs Superchip-aware casting for mixed-precision training (§ 4.5). Fifth, we develop a highly optimized Adam optimizer, GraceAdam, to make the best use of the Grace CPU (§ 4.6). Finally, PRISM can be extended with ZeRO-style data parallelism and Ulysses-style sequence parallelism to support multi-Superchip training and training with long sequences (§ 4.7).

4.2 Adaptive Weight-Stationary and Weight-Flow Offloading

When it comes to offloading LLM training, a key issue is to determine what model states to offload from GPU to CPU. While CPUs offer large memory capacity, transferring model

states to and from CPU memory still incurs time delays. Given that both the forward propagation and backward propagation of LLM training have a computation complexity of $O(bsz \cdot \Psi)$ and the optimizer state update has a complexity of $O(\Psi)$, where bsz is the batch size, state-of-the-art offloading systems often offload optimizer states and updates to the CPU [51, 54], which significantly reduces GPU memory consumption while avoiding offloading compute-intensive operations to the CPU.

However, an offloading decision must also be made for the model weights. Prior work often takes one of two directions: (1) *weight-stationary*, which keeps model weights on GPU [54], and (2) *weight-flow*, which partially loads model weights part-by-part [51]. For example, ZeRO-Offload keeps FP16 weights stationary on the GPU while offloading optimizer states to the CPU, reducing GPU memory consumption by roughly 8×. Conversely, ZeRO-Infinity partially offloads FP16 weights to DRAM or NVMe, allowing larger models to be trained with a single GPU. Given the drastic change in Superchip architecture and emerging post-training scenarios like long-context extension, an important question arises: shall we offload the FP16 weights from Hopper GPU to the Grace CPU to scale up model training on Superchip?

To analyze the theoretical viability of offloading FP16 weights to Grace CPU on Superchip, we use the peak computational throughput ($peak_{tp}$), data movement bandwidth (bw) to estimate training efficiency. The efficiency metric can be derived as follows:

$$comp\_time = \frac{total\_computation}{peak_{tp}} \tag{1}$$

$$comm\_time = \frac{total\_data\_movement}{bw} \tag{2}$$

$$efficiency = \frac{comp\_time}{comp\_time + comm\_time} \tag{3}$$

For the forward pass in LLMs, the total computation is dominated by the operations in the linear layers. This can be approximated as a function of the number of parameters, sequence length, and batch size (i.e., $2 \times bsz \times seq \times params$). During the forward, the model parameters must be loaded from CPU to GPU at least once, resulting in $data\_movement$ of $2 \times params$ in bytes. To identify $peak_{tp}$, we use the achievable peak instead of the theoretical hardware peak.

To ensure that data movement can be completely overlapped with computation, the efficiency should exceed 50% and ideally surpass 60% when considering communication latency and other overhead. Fig. 6 shows that even with a theoretical peak uni-directional C2C bandwidth of 450 GB/s, the batch size needs to be larger or equal to 4 with a sequence length of 1024 to achieve an efficiency greater than 60%.

This analysis indicates that the optimal offloading strategy is scenario-dependent. When scaling model sizes, each GPU can only handle a small micro-batch size to prevent out-of-memory, as seen in training large-scale LLMs like Bloom [61], Megatron-LM [59], and Turing-NLG models [34], which often use a micro-batch size of 1 or 2. As batch size and sequence lengths grow, such as in post-training with long-context extension [9, 12], activation memory may become significantly larger than model states. For instance, a 7B-parameter model requires 112GB for model states but needs 2TB of memory for activations with a sequence length of 1 million tokens. When this happens, it becomes more economical to transfer FP16 weights between the Hopper GPU and Grace CPU. In PRISM, we support both weight-stationary and weight-flow policies and adaptively choose the offloading policy for various practical settings.

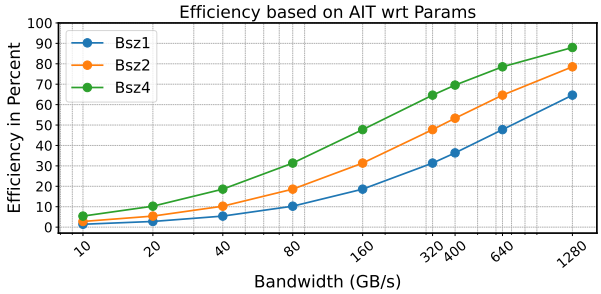

Figure 6. Impact of bandwidth on efficiency.

4.3 Fine-Grained Bucketization Repartitioning

One challenge in offloading-based solutions is that they cannot fully utilize both the computation and interconnect resources simultaneously because the GPU and CPU are idle during data movement operations. To overcome this challenge, prior work uses a *bucketization* strategy [54], dividing model states (e.g.,weights, gradients, optimizer states) into buckets to overlap GPU computation (e.g., MatMul during backward propagation) with swap-out/in operations (e.g., transferring gradients from GPU to CPU) in a fine-grained manner. This allows the swap-out/in operation of a bucket to start as soon as the compute kernel produces it.

In theory, bucketization should allow GPU computation to fully overlap with the optimizer step on CPU and data transfer between GPU and CPU. However, in practice, we observe that Hopper GPU still experiences long idle times with bucketization, as shown in Fig. 4. By diving deeper into the issue, we find that this is primarily because the FLOPS ratio between Hopper GPU and Grace CPU is ~330, which is significantly larger than prior architectures such as DGX-2 (~60.39) and DGX-A100 (~135.65). This increased compute gap makes balancing computation challenging. Take the last bucket as an example. In order for the last bucket's optimizer state to be executed on CPU, it requires a three-stage process: (i) gradient swap-out from GPU to CPU, (ii) execution of the Adam optimization algorithm on CPU, and (iii) swap-in the updated parameters back to the GPU. Since the first parameter in the forward pass is updated by the last gradient created in the backward pass, all of the above processes cannot overlap with any computation on GPU because the

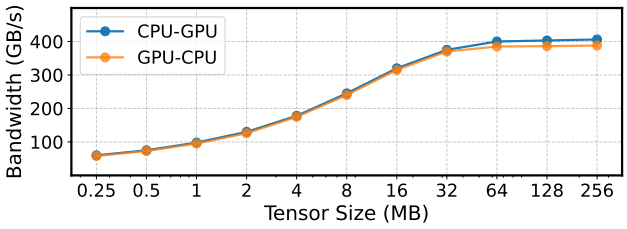

Figure 7. GH200 bandwidth measurement.

GPU needs to wait for the optimizer and weight updates to completely finish before starting the forward pass of the next training iteration. As a result, the latency from executing the last bucket(s) becomes exposed to the critical path length, and we need to carefully reorganize the computation to improve the Superchip utilization.

In contrast to previous studies [54], PRISM repartitions SA-DFG by placing optimizer states and gradients for the last few buckets on the GPU, provided they fit into Hopper GPU memory. Moreover, to overlap the compute and communication, the last bucket of gradients offloaded to the CPU must complete its optimization step and return updated parameters to the GPU before the next iteration begins. If we denote the number of buckets with optimizer states kept on the GPU as n, then:

$$\text{move}_{\text{bucket}}(grad) + \text{step}_{\text{cpu}}(bucket) + \text{move}_{\text{bucket}}(param) \leq \quad (4)$$

$$\text{bwd}(n \cdot bucket) + \text{step}_{\text{gpu}}(n \cdot bucket) \quad (5)$$

$\text{Step}_{\text{cpu/gpu}}(bucket)$ represents the time required to complete the optimizer step on the CPU/GPU for a bucket size of $bucket$. $\text{bwd}(n \cdot bucket)$ denotes the time to complete the backward pass for n buckets on GPU, which can be approximated as a function of the number of parameters, sequence length, and batch size, given by $4 \times bsz \times seq \times bk \times n/4$.

The analysis indicates that the bucket size bk is a crucial system parameter. Fig. 7 shows the bandwidth between GPU and CPU across different tensor sizes. It shows that the C2C bandwidth increases with tensor size until saturation occurs at approximately 64 MB. Based on this observation, we implement a bucketization strategy where gradients and parameters are grouped into buckets of 64 MB before transfer between GPU and CPU. This approach ensures efficient utilization of the available C2C bandwidth while also providing a convenient unit that enables fine-grained overlapping between compute and communication. Finally, PRISM determines the number of buckets to retain on GPU to hide Grace CPU execution and data transfer time. The optimal number of buckets depends on model size and batch sizes, and PRISM uses grid search to identify the optimal number.

4.4 Speculation-then-Validation (STV)

In most offloading solutions, synchronizations between CPU and GPU are needed in optimizer step to ensure numerical robustness. As an example, the clipping of the gradient norm requires calculating the global gradient norm [44], and mixed precision training requires a global check of NAN and INF values [33]. Both examples require the CPU to wait until all gradients have been received before the optimizer step and weight updates. As illustrated by the gray block in Fig. 3, this dependency exposes the optimizer step to the critical path, preventing it from overlapping with the backward pass.

To address this limitation, we propose a *speculation-then-validation* algorithm, which largely bypasses these synchronizations while preserving the exact convergence property of the training. Our mechanism is based on a key observation: *most of the time the global states have no effects*. For example, gradient clipping is rarely triggered, especially after the initial warm-up phase when gradient variance significantly reduces [29]. Similarly, mixed precision training rarely encounters NAN and INF, as a healthy training run should not have numerical instability issues.

Based on the observation, we propose to replace the conventional offloading synchronization-then-execution (STE) scheme with a speculation-then-validation (STV) mechanism. Fig. 8 illustrates the idea. Instead of waiting for all gradients to arrive, the CPU initiates the optimizer step speculatively using the gradients available at that moment. Meanwhile, gradient clipping and normalization are deferred to occur during CPU idle periods, concurrent with the GPU executing the forward pass for the next iteration. Meanwhile, to guarantee the synchronous nature of the optimization process, we implement the *in-place rollback* for the optimization step.

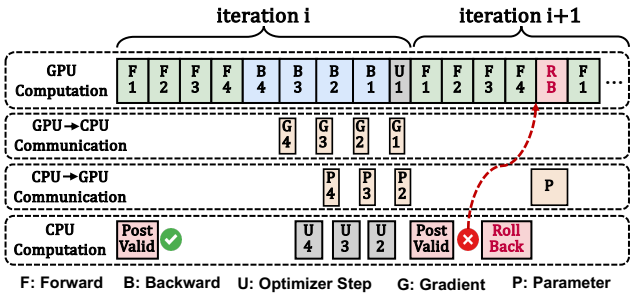

Figure 8. PRISM speculation-then-validation schedule. It enables optimizer steps (U) to overlap with backward propagation (B), eliminating synchronization bottlenecks and improving resource utilization.

The speculation-then-validation scheduling offers two key benefits. First, it allows the optimizer steps on the CPU to overlap with backward propagation on the GPU, rather than placing all optimizer steps sequentially on the critical path, which would significantly hurt throughput. Second, it eliminates the time cost of normalization calculations and value checking by relocating these operations to previously idle CPU cycles (when the GPU is executing the forward pass), thereby improving overall resource utilization. Compared with the ZeRO-Offload schedule in Fig. 3, PRISM reduces the

critical path of a single iteration from

$$T_{\text{iter}} = \text{fwd}_{\text{gpu}} + \text{bwd}_{\text{gpu}} + \text{move}_{\text{bucket}}(grad) + \text{step}_{\text{cpu}} + \text{move}_{\text{bucket}}(param) \quad (6)$$

to

$$T_{\text{iter}} = \text{fwd}_{\text{gpu}} + \text{bwd}_{\text{gpu}} + \text{step}_{\text{cpu}}(bucket) \quad (7)$$

Interestingly, in theory, the STV schedule indicates that the iteration time of PRISM can be shorter than that of non-offload training because the optimizer steps are performed on the entire model using GPU in non-offload training, while PRISM only updates a subset of the parameters on GPU. However, the conventional wisdom is that offloading often comes at a performance cost. In our evaluation, we show that PRISM can indeed be faster than GPU-only solutions.

4.5 Superchip-Aware Casting (SAC)

In DL training frameworks like PyTorch and DeepSpeed, the mixed precision training is implemented through a graph rewriting process [46]. The default precision of all ops is float32 (FP32). Mixed precision training casts certain model states (e.g., weights, gradients) from FP32 to float16 (FP16), or vice versa. For example, the gradients in the backward pass are produced in FP16, and the optimizer computes the updates using FP32 gradients. Therefore, the functions $\text{move}_{\text{bucket}}(grad)$ and $\text{move}_{\text{bucket}}(param)$ not only transfer tensors between the GPU and CPU but also involve converting tensor data types. These additional casting operations become complex when considering offloading strategies. Existing offloading-based solutions often adopt a minimum edge cut algorithm to computation graph [54], based on the implicit assumption that minimizing the data communication volume between the CPU and GPU leads to performance improvements. However, these methods ignore the casting cost. Given that the C2C link between Hopper and Grace is an order of magnitude higher, is this assumption still valid?

We conduct experiments to examine the time cost of the two methods. The results are shown in Fig. 9. We observe that $Cast_{cpu} \leftrightarrow Move_{fp16}$ takes around 2× execution time than $Cast_{gpu} \leftrightarrow Move_{fp32}$ on Superchips, depending on the size of the input tensor involved in mixed precision training. In most cases, $Cast_{gpu} \leftrightarrow Move_{fp32}$ surpasses the performance of $Cast_{cpu} \leftrightarrow Move_{fp16}$ by a large margin (e.g., tensor size 256MB to 2048MB). Based on our analysis, we opt for $Cast_{gpu} \leftrightarrow Move_{fp32}$, as it incurs an overall lower cost compared to $Cast_{cpu} \leftrightarrow Move_{fp16}$ even though the latter design incurs a lower theoretical communication volume.

In addition to the casting cost, we also find that the casting operation has an intricate interaction with the pinned memory. A transfer-then-cast operation triggers an unpinned temporary memory buffer allocation on the Grace CPU to hold the FP16 copy of gradients swapped-out from the Hopper GPU before casting FP16 gradients to FP32 on the Grace CPU. As such, the data transfer is implicitly through unpinned memory, which is significantly slower than DMA transfer with FP32. This observation also motivates us to adopt the $Cast_{gpu} \leftrightarrow Move_{fp32}$ design in PRISM.

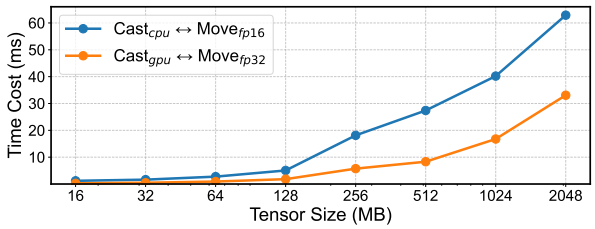

Figure 9. Time cost comparison for casting operations on GPU vs. CPU (including data transfer overhead).

4.6 GraceAdam

The careful reader may ask: Is Grace CPU in a Superchip sufficient for optimizer step and weight updates given the increased GPU/CPU FLOPS ratio? Prior work introduces CPU-Adam [54], a highly-optimized implementation to accelerate Adam optimizer on CPU through multi-level parallelism.

However, the original CPU-Adam implementation was primarily designed for x86 architectures using AVX2/AVX512 instruction sets, which are not available on ARM CPUs. Moreover, while x86 implementations rely on fixed-width SIMD vectors, ARM's Scalable Vector Extension (SVE) employs a length-agnostic vector processing paradigm that requires more adaptive algorithms. Finally, ARM CPU has a different memory hierarchy, with different cache sizes, latencies, and bandwidth characteristics, which requires careful consideration of memory access patterns and prefetching strategies.

To address this limitation, we introduce GraceAdam, which includes multiple optimizations: 1) **SVE Integration:** We adopt SVE's length-agnostic programming model to dynamically determine vector length at runtime with `svcntw()`. We replace scalar operations with SVE-specific intrinsics such as `svld1_f32`, `svmla_f32_m`, and `svsqrt_f32_m` that operate on entire vectors simultaneously and thus reduce the computation cost. 2) **Enhanced Memory Management:** We implement ARM-specific memory access patterns with explicit prefetching strategies using `svprfm` directives that preload data into cache before computation. Our tiled processing approach divides parameter updates into cache-friendly chunks (TILE size), minimizing data movement between memory hierarchies. We further optimize memory access by aligning SVE load/store operations (`svld1_f32`, `svst1_f32`) with ARM's cache line boundaries, reducing cache thrashing during parameter updates. 3) **Parallel Execution:** We employ a dual-level parallelism strategy, where we use OpenMP multi-threading to ensure efficient scaling across multi-core ARM Grace CPUs and SVE for instruction-level parallelism within each core. These strategies together allow GraceAdam to maximize Grace CPU throughput and achieve significantly faster speed than the default PyTorch Adam on Grace CPU.

4.7 Multi-Superchip Schedule

In this part, we describe how PRISM can be extended to work effectively in a multi-Superchip environment by combining it with ZeRO-style data parallelism (ZeRO-DP) [50] and Ulysses-style sequence parallelism (Ulysses-SP) [21].

ZeRO-3 Integration. PRISM preserves the model state partitioning strategy of ZeRO stage-3 (Fully Sharded Data Parallelism). Each GPU offloads a portion of its gradients and optimizer states to CPU memory while the partitioned weights, as well as the last few buckets from adaptive offloading (§ 4.2) remain on the GPUs. The primary advantage of partitioning before offloading is that in a multi-Superchip system, each parallel process is responsible for updating only a subset of the model parameters. This design ensures that the total communication volume from all GPUs to the CPU remains constant, while the CPU resources are utilized in parallel to jointly compute a unified weight update.

PRISM-Ulysses. PRISM can also work together with Ulysses-SP. Recent advancements in large language models have demonstrated a clear trend toward training and fine-tuning with increasingly longer sequence lengths. Notable examples include Claude's expansion of context window from 100K to 200K tokens [3] and GPT-4 Turbo's increase from 8K to 128K tokens [42]. This evolution is driven by real-world applications that require processing extensive documents, images, and videos, requiring models capable of handling substantially longer sequences.

Ulysses-SP [21] addresses sequence parallelism challenges by partitioning input data along the sequence dimension and employing an efficient all-to-all collective communication strategy for attention computations. However, even with activation checkpointing, the fixed GPU memory consumption of model states significantly limits the ability to scale to longer sequences. We integrate Ulysses-SP with PRISM by leveraging its offloading capabilities to manage memory more efficiently. Specifically, PRISM's adaptive weight-flow policy offloads optimizer states and the majority of model weights to CPU memory, significantly reducing the GPU memory footprint. This enables Ulysses-SP to handle longer sequences on a per-layer basis without being constrained by GPU memory limitations. We term the combined strategy *PRISM-Ulysses*. PRISM-Ulysses supports 8× longer sequence training than Ulysses-SP and easily enables million-token sequence length LLM training with high throughput using only a few Superchips. See § 5.3 for more details.

NUMA binding. In a K-way Superchip node, there are K Hopper GPUs and K Grace CPUs, and each Superchip is configured as a distinct NUMA node. Assume K = 4, a 4-way Superchip node has a total of 288 CPU cores (e.g., each Grace CPU has 72 cores). By default, a training launcher may randomly assign the process to certain CPU cores, which leads to issues such as the CPU process and GPU are not co-located on the same Superchip. If this happens, data transfers traverse an inter-Superchip connection (typically Slingshot), which has significantly lower bandwidth compared to the NVLINK-C2C connection. To mitigate this performance penalty, PRISM explicitly binds each process to specific cores to ensure optimal CPU–GPU affinity.

5 Evaluation

In this section, we conduct a comprehensive evaluation of PRISM in comparison with state-of-the-art approaches, demonstrating that it achieves excellent training efficiency and scalability across various model sizes and extremely long sequence training. We also show the impact of different technologies within PRISM on overall performance.

5.1 Evaluation Methodology

Hardware. We conducted single Superchip experiments on a *1×GH200* (96GB HBM, 480GB DDR). For multiple nodes experiments, we used 8 *GH200 NVL2* nodes, each containing *2×GH200* (96GB HBM, 240GB DDR), and the nodes are connected via HPE/Cray's 200Gbps Slingshot 11 interconnect.
Workloads. For the performance evaluation, we focus on evaluating GPT/LLaMA [48, 64] like Transformer-based models [65]. We vary the hidden dimension and the number of Transformer blocks to obtain models with a different number of parameters. Appendix A shows the configuration parameters used in our experiments. We use a subset of the Pile dataset [11] for training datasets.
Baseline. We compare the effectiveness of PRISM with state-of-the-art multi-billion parameter training solutions: PyTorch DDP [26], which uses data parallelism across GPUs; Megatron [59], which employs model parallelism; ZeRO-2/3 [50], which shard gradients, optimizer states, and parameters (ZeRO-3) across GPUs; ZeRO-Offload [54], which extends ZeRO-2 by offloading optimzer states to CPU; ZeRO-Infinity [51], an extension of ZeRO-3 that offloads model states to CPU and NVMe (we only enable its CPU offloading for fair comparison); and FSDP-CPU Offload [72], which extends FSDP by offloading model states to CPU. A more detailed description of baselines is provided in Appendix B.

5.2 Training Throughput

Single Superchip. We compare the training throughput of PRISM with PyTorch DDP, FSDP-Offload, ZeRO-Infinity and ZeRO-Offload on a single Superchip across different model sizes. We exclude Megatron and ZeRO-2/3 from the comparison because they do not change the workload on a single GPU compared to PyTorch DDP, and none of them can train models exceeding 4B parameters due to out-of-memory (OOM) errors. We evaluate them on the same batch size. And when large batch sizes cause OOM errors, we employ two strategies: (1) using smaller micro-batch sizes with gradient accumulation, or (2) implementing activation checkpointing

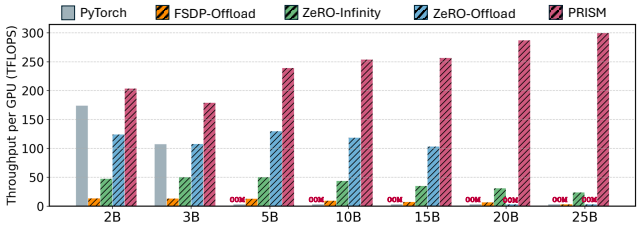

Figure 10. The training throughput with PyTorch DDP, FSDP-Offload, ZeRO-Infinity, ZeRO-Offload, and PRISM on a single Superchip with batch size 8.

with the largest micro-batch size without OOM errors. Note that we exclude recomputation volume when calculating TFLOPS to measure effective computational throughput. For each techniques, we report the higher throughput achieved between these two approaches.

Fig. 10 shows the comparison results. Interestingly, PRISM not only outperforms existing offloading methods, but it also outperforms GPU-only approaches across all tested model sizes, which is different from conventional wisdom where offloading often comes at the cost of performance penalty. Specifically, PRISM achieves up to 67% higher throughput (TFLOPS) compared to PyTorch DDP. The reasons are two folds: (1) PRISM offloads the optimizer steps to CPU, fully overlapping them with GPU backward propagation, and (2) by offloading the optimizer states to CPU memory, GPU has more memory space to hold the activations for larger batch without requiring activation checkpointing, which typically reduces throughput by approximately 33% [7]. Moreover, PRISM achieves 2× throughput on average (up to 2.5×) compared to ZeRO-Offload. The performance benefit of PRISM comes from three aspects. First, the speculation-then-validation (§ 4.4) algorithm enables effective overlapping of CPU and GPU computation, whereas in ZeRO-Offload, CPU optimizer steps must wait until GPU backward propagation completes. Second, the bucketization repartitioning (§ 4.3) eliminates communication bottlenecks on the critical path when transferring gradients and updated parameters, while in ZeRO-Offload, the forward pass for the next iteration must wait for updated parameters to be completely transferred from CPU to GPU. Third, GraceAdam (§ 4.6) further reduces the computation cost of optimizer steps on Grace CPU.

PRISM achieves significantly higher throughput than both FSDP-Offload and ZeRO-Infinity. FSDP-Offload consistently achieves less than 15 TFLOPS because it relies on PyTorch's native CPU-Adam implementation. As we demonstrate later in § 5.6, native implementation is up to 3.5× slower than our GraceAdam, creating a substantial efficiency bottleneck. Similarly, ZeRO-Infinity's throughput remains below 50 TFLOPS. This limitation stems from ZeRO-Infinity's bucket-based approach for transferring gradients and parameters between CPU and GPU, which fails to account for the characteristics of C2C bandwidth—specifically, that bandwidth can drop to as low as 50GB/s with small tensor sizes.

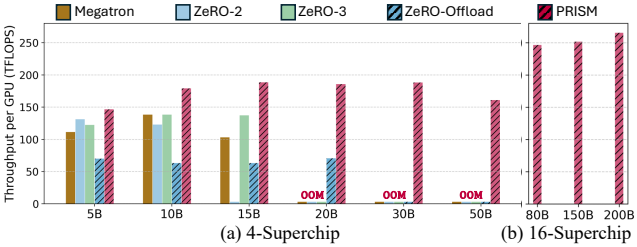

Figure 11. Training throughput comparison between Megatron, ZeRO-2/3, ZeRO-Offload, and PRISM on 4 and 16 GH200 GPUs. We use batch sizes of 16 and 128 for the 4-GPU and 16-GPU experiments, respectively.

Multiple Superchips. We evaluate the training throughput of PRISM against Megatron, ZeRO-3, ZeRO-3, and ZeRO-Offload across 4 and 16 GH200 Superchips. We exclude the PyTorch DDP from the comparison as it cannot scale out to larger model sizes, and we exclude FSDP-offload and ZeRO-Infinity due to their significantly lower throughput compared to other solutions on single Superchip. For Megatron, we use a MP degree that gives the best performance. Similar to the above single GPU experiment, when facing OOM errors, we either use gradient accumulation with smaller micro-batches or activation checkpointing with the largest possible micro-batch, reporting the higher throughput between these approaches. Fig. 11 shows the throughput per GPU results when training on 4 Superchips and 16 Superchips. We have the following observations:

- For models ranging from 5B to 20B parameters, PRISM consistently achieves higher throughput compared to existing solutions. Specifically, it demonstrates throughput improvements of up to 83%, 46%, and 37% over Megatron, ZeRO-2, ZeRO-3 respectively. In comparison with ZeRO-Offload, PRISM achieves 2.5× throughput on average across a wider range of model sizes. This is primarily attributed to highly overlapped CPU and GPU computation and sufficient GPU memory space for larger micro batch without using activation checkpointing, and those methods effectively leverage the high-bandwidth C2C interconnect in the Superchip architecture, enabling more efficient utilization of both GPU and CPU resources during training.
- While Megatron and ZeRO-3 encounter memory limitations at 15B parameters due to insufficient aggregate GPU memory across 4 GPUs, PRISM effectively scales to models with up to 50B parameters. Moreover, PRISM demonstrates exceptional scalability by efficiently training 200B models on 16 GPUs while maintaining high computational throughput.

5.3 Unlock Massive Sequence Training

We combine PRISM and Ulysses [21] to create PRISM-Ulysses for long-sequence training and evaluate its performance against vanilla Ulysses using two commonly deployed model

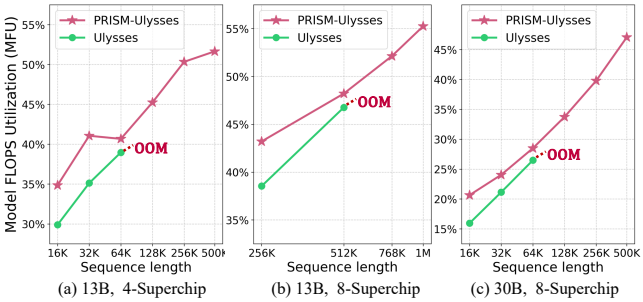

Figure 12. Supported sequence lengths and corresponding MFU using PRISM-Ulysses and Ulysses. OOM denotes the point where increasing sequence length causes OOM.

sizes: 13B and 30B parameters. Our experiments are conducted on both 4-Superchip and 8-Superchip. For all experiments, we utilize the maximum batch sizes that avoid out-of-memory errors, and implemented activation checkpointing when activation sizes are substantial. As shown in Fig. 12, PRISM-Ulysses trains sequences up to 8× longer than Ulysses. PRISM-Ulysses enables the training of 13B model with sequence lengths up to 1 million tokens on 8 Superchips while achieving 55% MFU. For sequence lengths that Ulysses can handle, PRISM-Ulysses consistently achieves higher Model FLOPS Utilization (MFU). This performance advantage stems from our adaptive Offloading policy (§ 4.2), which adaptively detects extremely long sequences and dynamically offloads more weight parameters to CPU memory. This strategic offloading provides GPUs with additional memory space to accommodate larger activations without requiring activation checkpointing, thereby maintaining computational efficiency even with extended sequence lengths.

5.4 Model Scale

In this part, we first test the largest trainable models on a single Superchip as well as 4-Superchip and 16-Superchip.

Single Superchip. Fig. 13 shows that the largest model can be trained using PyTorch DDP on a single GPU with 96GB memory is 3.5B. Megatron, ZeRO-2, and ZeRO-3 do not enable training larger models on a single GPU compared to PyTorch DDP, because they all depend on the aggregated GPU memory to store model states. ZeRO-Offload, however, can handle models up to 15B by offloading optimizer states. PRISM can train models up to 25B – primarily because it adaptively controls the offloading ratio for optimizer states and gradients. Unlike ZeRO-Offload, which keeps weights stationary on GPUs and uses a fixed pattern for tensor placement across devices, PRISM adaptively adopts weight-stationary and weight-flow policy based on the available GPU and CPU memory, thereby increasing the trainable model size. While ZeRO-Infinity can train models of comparable size to PRISM, its throughput is significantly lower. As shown in Fig. 10, PRISM achieves on average 6.7× higher throughput (up to 12.6×) than ZeRO-Infinity. This is mainly

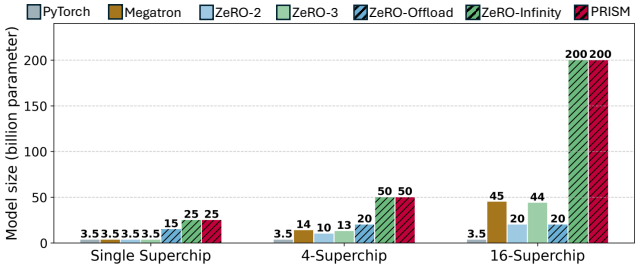

Figure 13. The size of the biggest model that can be trained on single Superchip, 4 and 16 Superchips.

due to the technologies introduced by PRISM such as STV and bucketization repartitioning that improve the overall utilization of the Hopper GPU and C2C bandwidth.

Multiple Superchips. We further perform model scale tests with 4 and 16 GPUs in a single GH200 Node and four GH200 nodes, respectively. As shown in Fig. 13, the maximum trainable model size stays the same for PyTorch DDP, because none of them handle memory redundancies in data parallelism. As a result, its scalability is bounded by the model scale on a single GPU. Megatron, ZeRO2/3 support large model training with more GPUs, but they cannot scale efficiently beyond 50B parameters, even with 16 GPUs. Megatron supports larger models than ZeRO-2, because ZeRO-2 still incurs memory redundancies on model weights. Since ZeRO-Offload is built on top of ZeRO-2, it needs each GPU to hold at least the full copy of FP16 parameter, therefore, the largest model size the ZeRO-Offload can train is bounded to 20B even though we increase the number of GPUs. Overall, PRISM increases the model scale on 16 Superchips by 57×, 4.4×, 10×, 4.5×, 10× than using PyTorch DDP, Megatron, ZeRO-2, ZeRO-3, and ZeRO-Offload, respectively.

5.5 Performance Breakdown

We systematically enable each optimization component to quantify its individual contribution to overall performance. We use the same setting as used in § 5.2 and use 5B model as a demonstration. Table 2 presents a comprehensive breakdown of PRISM's key optimizations and their cumulative impact on training throughput. Starting with the baseline configuration (all optimizations disabled), we observe a throughput of 116.2 TFLOPS, that is close to the ZeRO-Offload throughput shown in Fig. 10. Enabling GraceAdam increases throughput to 128.23 TFLOPS, representing a 10.4% improvement. When further enabling SAC from § 4.5, throughput increases to 144.49 TFLOPS, delivering an additional 12.7% improvement. This optimization strategically balances casting and data transfer costs for maximum efficiency. The Speculation-Then-Validation (STV) technique described in § 4.4 delivers the most substantial performance gain, boosting throughput by 45%. This significant enhancement demonstrates how effectively STV eliminates Hopper GPU computational idle

Table 2. Breakdown of PRISM optimizations and their impact on throughput (TFLOPS), including GraceAdam (§ 4.6), Superchip-Aware Casting (SAC, § 4.5), Speculation-then-Validation (SVT, § 4.4), and Bucket Repartitioning (§ 4.3).

GraceAdam	Cast Optim.	STV	Buck. Repart.	Throughput
✗	✗	✗	✗	116.20
✓	✗	✗	✗	128.23
✓	✓	✗	✗	144.49
✓	✓	✓	✗	209.36
✓	✓	✓	✓	238.92

time by overlapping backward propagation with Grace CPU-based optimization steps. Finally, enabling the bucketization repartitioning from § 4.3 further increases throughput to 238.92 TFLOPS, representing a 14.1% improvement. In total, PRISM with all optimizations enabled achieves a 2.06× throughput improvement over the baseline configuration. This breakdown highlights how each component addresses specific performance bottlenecks in large-scale training on Superchip architectures, with their combined effect delivering substantial multiplicative benefits.

5.6 Optimized GraceAdam Execution

Table 3 evaluates GraceAdam by measuring the optimizer execution times across model sizes ranging from 1B to 8B. Compared to PyTorch's CPU implementation (PT-CPU), GraceAdam achieves over 3× speedup across all configurations. Even compared to the already-optimized CPU-Adam implementation, GraceAdam demonstrates 1.36× faster execution. These performance gains from GraceAdam's optimizations that leverage ARM's Scalable Vector Extension (SVE), along with optimized memory management and OpenMP multithreading, which make Grace CPU a viable and performant component within the PRISM system.

Table 3. Adam latency (s) for PyTorch Native CPU-Adam, CPU-Adam, and GraceAdam.

#Parameter	PT-CPU	CPU-Adam	GraceAdam
1 billion	0.289	0.098	0.082
2 billion	0.531	0.198	0.160
4 billion	0.958	0.393	0.316
8 billion	1.834	0.769	0.608

5.7 Speculation-Then-Validation Evaluation

Speculation-then-Validation in § 4.4 is an exact optimization but the schedule is complex. To validate its correctness and overhead, we trained a GPT-style 175B model with the Pile dataset on 16 Superchips. Note that a GPT-style 175B model often requires thousands of GPUs to train [4, 61, 70], which are simply out of reach for most model scientists. Figure 14 shows the pre-training loss curve over 80,000 training iterations, with the expected convergence trend. Though from iterations 1 to 1000, roll-backs occur frequently as gradient clipping and INF & NaN values are mainly detected during the initial training stage. After iteration 1000, when training becomes more stable, roll-backs rarely happen - occurring only 93 times between steps 1000 and 80000, which represents 0.12% of the total iterations. For the 175B model, roll-back operations executed in parallel across 16 Grace CPUs take 2 seconds on average, accumulating to less than 200 seconds over 79,000 iterations. This represents a negligible overhead while fully preserving training accuracy.

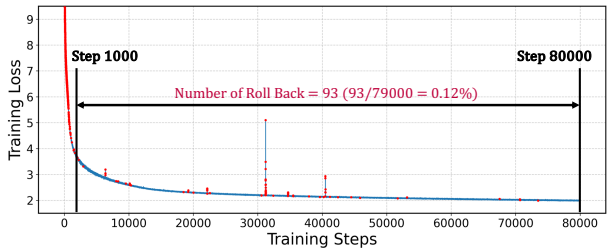

Figure 14. Training loss and rollback occurrences during training of the GPT 176B model over 80,000 iterations. Red dots indicate iterations where rollbacks were triggered due to gradient clipping, NaN or INF values.

5.8 GPU Idle Time Measurement

Unlike the prior solutions cause GPU idle time (as shown in Fig. 4), we measure the GPU utilization with the same model and hardware setting. As shown in Fig. 15, PRISM achieves near-complete GPU utilization, effectively eliminating idle periods and maximizing computational efficiency.

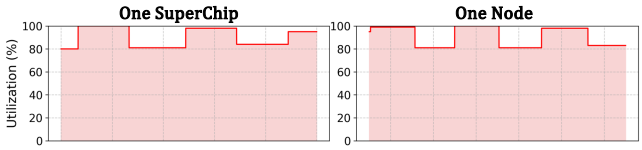

Figure 15. PRISM fully utilizes the GPU resources.

6 Conclusion

In this paper, we have taken a leap forward in designing a Superchip-centric offloading system for large-scale LLM training. Our analysis reveals key challenges and performance implications of training LLMs on Superchips. Motivated by the observations, we design PRISM, the first system that allows LLM training to make the best use of Hopper GPU, Grace CPU, and NVLink-C2C interconnects simultaneously. Our extensive evaluation demonstrates that PRISM not only outperforms state-of-the-art offloading-based solutions but also unlocks new system capabilities such as training LLMs with 1 million token sequence length with only 8 Superchips, making large-scale LLM training more accessible to researchers and practitioners even without a large number of GPUs.

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Qin Cai, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Weizhu Chen, Yen-Chun Chen, Yi-Ling Chen, Hao Cheng, Parul Chopra, Xiyang Dai, Matthew Dixon, Ronen Eldan, Victor Fragoso, Jianfeng Gao, Mei Gao, Min Gao, Amit Garg, Allie Del Giorno, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Wenxiang Hu, Jamie Huynh, Dan Iter, Sam Ade Jacobs, Mojan Javaheripi, Xin Jin, Nikos Karampatziakis, Piero Kauffmann, Mahoud Khademi, Dongwoo Kim, Young Jin Kim, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Yunsheng Li, Chen Liang, Lars Liden, Xihui Lin, Zeqi Lin, Ce Liu, Liyuan Liu, Mengchen Liu, Weishung Liu, Xiaodong Liu, Chong Luo, Piyush Madan, Ali Mahmoudzadeh, David Majercak, Matt Mazzola, Caio César Teodoro Mendes, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Liliang Ren, Gustavo de Rosa, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacroce, Shital Shah, Ning Shang, Hiteshi Sharma, Yelong Shen, Swadheen Shukla, Xia Song, Masahiro Tanaka, Andrea Tupini, Praneetha Vaddamanu, Chunyu Wang, Guanhua Wang, Lijuan Wang, Shuohang Wang, Xin Wang, Yu Wang, Rachel Ward, Wen Wen, Philipp Witte, Haiping Wu, Xiaoxia Wu, Michael Wyatt, Bin Xiao, Can Xu, Jiahang Xu, Weijian Xu, Jilong Xue, Sonali Yadav, Fan Yang, Jianwei Yang, Yifan Yang, Ziyi Yang, Donghan Yu, Lu Yuan, Chenruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. *arXiv preprint arXiv:2404.14219* (2024).

[2] AMD. 2024. AMD Instinct MI300A. https://www.amd.com/en/products/accelerators/instinct/mi300/mi300a.html.

[3] Anthropic. 2024. Claude. https://www.anthropic.com/claude.

[4] Anyscale. 2023. Training 175B Parameter Language Models at 1000 GPU scale with Alpa and Ray. https://www.anyscale.com/blog/training-175b-parameter-language-models-at-1000-gpu-scale-with-alpa-and-ray.

[5] Microsoft Azure. 2024. Microsoft and NVIDIA partnership continues to deliver on the promise of AI. https://azure.microsoft.com/en-us/blog/microsoft-and-nvidia-partnership-continues-to-deliver-on-the-promise-of-ai.

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS'20)*.

[7] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. *arXiv preprint arXiv:1604.06174* (2016).

[8] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2024. LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models. *arXiv preprint arXiv:2309.12307* (2024).

[9] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. 2024. LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens. *arXiv preprint arXiv:2402.13753* (2024).

[10] Luigi Fusco, Mikhail Khalilov, Marcin Chrapek, Giridhar Chukkapalli, Thomas Schulthess, and Torsten Hoefler. 2024. Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip. *arXiv preprint arXiv:2408.11556* (2024).

[11] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. *arXiv preprint arXiv:2101.00027* (2020).

[12] Tianyu Gao, Alexander Wettig, Howard Yen, and Danqi Chen. 2024. How to Train Long-Context Language Models (Effectively). *arXiv preprint arXiv:2410.02660* (2024).

[13] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *arXiv preprint arXiv:2403.14123* (2024).

[14] Mark Hildebrand, Jawad Khan, Sanjeev Trika, Jason Lowe-Power, and Venkatesh Akella. 2020. AutoTM: Automatic Tensor Movement in Heterogeneous Memory Systems using Integer Linear Programming. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*.

[15] HPCwire. 2024. HPE's New Helios Supercomputer Elevates Polish Scientific Research at Cyfronet. https://www.hpcwire.com/off-the-wire/hpes-new-helios-supercomputer-elevates-polish-scientific-research-at-cyfronet.

[16] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*.

[17] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, and et al. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NIPS'19)*.

[18] J. Alex Hurt, Grant J. Scott, Derek Weitzel, and Huijun Zhu. 2024. Adventures with Grace Hopper AI Super Chip and the National Research Platform. *arXiv preprint arXiv:2410.16487* (2024).

[19] Information Technology Center, The University of Tokyo. 2024. Miyabi Supercomputer System. https://www.cc.u-tokyo.ac.jp/en/supercomputer/miyabi/system.php.

[20] insideHPC. 2024. Eviden Delivers 104 PFLOPS NVIDIA-Powered EXA1-HE Supercomputer for CEA. https://insidehpc.com/2024/04/eviden-delivers-104-pflops-nvidia-powered-exa1-he-supercomputer-for-cea.

[21] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. *arXiv preprint arXiv:2309.14509* (2023).

[22] Jülich Supercomputing Centre. 2024. JUPITER Supercomputer. https://www.fz-juelich.de/en/ias/jsc/jupiter/tech.

[23] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361* (2020).

[24] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR'15)*.

[25] Lambda. 2024. NVIDIA GH200 Grace Hopper Superchip. https://lambdalabs.com/nvidia-gh200.

[26] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, and et al. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *arXiv preprint*

arXiv:2006.15704 (2020).

[27] Xinyu Lian, Yinfang Chen, Runxiang Cheng, Jie Huang, Parth Thakkar, Minjia Zhang, and Tianyin Xu. 2025. Large Language Models as Configuration Validators. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE'25)*.

[28] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring Attention with Blockwise Transformers for Near-Infinite Context. *arXiv preprint arXiv:2310.01889* (2023).

[29] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. 2020. On the Variance of the Adaptive Learning Rate and Beyond. In *Proceedings of the 8th International Conference on Learning Representations (ICLR'20)*.

[30] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR'19)*.

[31] Avinash Maurya, Jie Ye, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. Deep Optimizer States: Towards Scalable Training of Transformer Models using Interleaved Offloading. In *Proceedings of the 25th International Middleware Conference (Middleware'24)*.

[32] Meta AI LLaMA Team. 2024. The Llama 3 Herd of Models. https://ai.meta.com/research/publications/the-llama-3-herd-of-models.

[33] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *Proceedings of the 6th International Conference on Learning Representations (ICLR'18)*.

[34] Microsoft Research. 2020. Turing-NLG: A 17-billion-parameter language model by Microsoft. https://www.microsoft.com/en-us/research/blog/turing-nlg-a-17-billion-parameter-language-model-by-microsoft.

[35] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'19)*.

[36] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *arXiv preprint arXiv:2104.04473* (2021).

[37] National Center for Supercomputing Applications. 2024. Delta-AI. https://delta.ncsa.illinois.edu/delta-delta-ai.

[38] NVIDIA. 2022. NVIDIA DGX A100: The Universal System for AI Infrastructure. https://images.nvidia.com/aem-dam/Solutions/Data-Center/nvidia-dgx-a100-datasheet.pdf.

[39] NVIDIA. 2023. NVIDIA Grace Hopper Superchip Architecture In-Depth. https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth.

[40] NVIDIA. 2024. NVIDIA GB200 NVL72. https://www.nvidia.com/en-us/data-center/gb200-nvl72.

[41] NVIDIA. 2024. NVIDIA Grace Hopper Superchip. https://www.nvidia.com/en-us/data-center/grace-hopper-superchip.

[42] OpenAI. 2023. GPT-4. https://openai.com/product/gpt-4.

[43] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. *arXiv preprint arXiv:2203.02155* (2022).

[44] Razvan Pascanu, Tomás Mikolov, and Yoshua Bengio. 2013. On the Difficulty of Training Recurrent Neural Networks. In *Proceedings of the 30th International Conference on Machine Learning, (ICML'13)*.

[45] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*.

[46] PyTorch Team. 2025. Automatic Mixed Precision. https://pytorch.org/tutorials/recipes/recipes/amp_recipe.html.

[47] PyTorch Team. 2025. FSDP Tutorial. https://pytorch.org/tutorials/intermediate/FSDP_tutorial.html.

[48] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models are Unsupervised Multitask Learners*. Technical Report. OpenAI.

[49] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *arXiv preprint arXiv:2305.18290* (2024).

[50] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2019. ZeRO: Memory Optimization Towards Training A Trillion Parameter Models. *arXiv preprint arXiv:1910.02054* (2019).

[51] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. *arXiv preprint arXiv:2104.07857* (2021).

[52] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD'20)*.

[53] Jie Ren, Jiaolin Luo, Kai Wu, Minjia Zhang, Hyeran Jeon, and Dong Li. 2021. Sentinel: Efficient Tensor Migration and Allocation on Heterogeneous Memory Systems for Deep Learning. In *Proceedings of the 27th International Symposium on High-Performance Computer Architecture (HPCA'21)*.

[54] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC'21)*.

[55] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-49)*.

[56] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2024).

[57] Gabin Schieffer, Jacob Wahlgren, Jie Ren, Jennifer Faj, and Ivy Peng. 2024. Harnessing Integrated CPU-GPU System Memory for HPC: a first look into Grace Hopper. *arXiv preprint arXiv:2407.07850* (2024).

[58] Amazon Web Services. 2023. NVIDIA Announces Availability of NVIDIA GH200 Grace Hopper Superchips on Amazon EC2. https://aws.amazon.com/solutions/case-studies/nvidia-keynote-aws-reinvent-2023.

[59] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2020).

[60] Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, Perry Payne, Martin Seneviratne, Paul Gamble,

Chris Kelly, Nathaneal Scharli, Aakanksha Chowdhery, Philip Mansfield, Blaise Aguera y Arcas, Dale Webster, Greg S. Corrado, Yossi Matias, Katherine Chou, Juraj Gottweis, Nenad Tomasev, Yun Liu, Alvin Rajkomar, Joelle Barral, Christopher Semturs, Alan Karthikesalingam, and Vivek Natarajan. 2023. Large Language Models Encode Clinical Knowledge. *arXiv preprint arXiv:2212.13138* (2023).

[61] BigScience team. 2023. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv preprint arXiv:2211.05100* (2023).

[62] Gemini Team. 2024. Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805* (2024).

[63] TOP500. 2024. Alps Supercomputer. https://top500.org/system/180259.

[64] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).

[65] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS'17)*.

[66] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. SuperNeurons: Dynamic GPU Memory Management for Training Deep Neural Networks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'18)*.

[67] xAI. 2024. Grok-2. https://x.ai/blog/grok-2.

[68] Yi Xu, Ziming Mao, Xiangxi Mo, Shu Liu, and Ion Stoica. 2024. Pie: Pooling CPU Memory for LLM Inference. *arXiv preprint arXiv:2411.09317* (2024).

[69] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, and Guoyin Wang. 2024. Instruction Tuning for Large Language Models: A Survey. *arXiv preprint arXiv:2308.10792* (2024).

[70] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).

[71] Yiqi Zhang and Yang You. 2024. SpeedLoader: An I/O Efficient Scheme for Heterogeneous and Distributed LLM Operation. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NIPS'24)*.

[72] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *arXiv preprint arXiv:2304.11277* (2023).

A Model Config

Table 4. Model configuration in evaluation.

# params	# layer	Hidden size
1, 2, 3 billion	20, 40, 60	2048
4 billion	64	2304
5, 6, 8 billion	44, 53, 72	3072
10, 11 billion	50, 55	4096
12, 13 billion	60, 65	4096
15 billion	78	4096
20, 25, 50, 60 billion	25, 30, 60, 75	8192
70, 80 billion	87, 100	8192
150, 200 billion	45, 60	16384

B Baseline

We compare the effectiveness of PRISM with state-of-the-art multi-billion parameter training solutions:

- **PyTorch DDP** [26]: The standard PyTorch Transformer implementation using DistributedDataParallel (DDP) for data parallelism across multiple GPUs.
- **Megatron** [59]: A widely used large-scale model training solution that employs model parallelism to efficiently scale Transformer models.
- **ZeRO-2** [50]: An extension of data parallelism that eliminates memory redundancies across multiple GPUs by sharding gradients and optimizer states.
- **ZeRO-3** [50]: An enhancement over ZeRO-2 that further shards model parameters across GPUs.
- **ZeRO-Offload** [54]: Built on top of ZeRO-2, ZeRO-Offload further offloads gradients and optimizer states to CPU memory, providing an efficient solution for training large-scale models with limited GPU memory. It has been adopted in DeepSpeed as the default offloading solution to CPU.
- **ZeRO-Infinity** [51]: An extension of ZeRO-3 and ZeRO-Offload that enables training very large models by utilizing GPU, CPU memory, and NVMe storage through optimized memory management, prefetching, and computation-communication overlapping. For fair comparison, we only enable its CPU offloading capabilities, excluding NVMe storage offloading.
- **FSDP-CPU Offload** [72]: Fully Sharded Data Parallel (FSDP) with CPU offloading is an advanced memory-saving technique that extends FSDP by offloading model states, including parameters, gradients, and optimizer states, to CPU when not actively needed.